マイコンシステム
第１２回

青森大学 ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

目次

- 講義
  - 内部設計③
  - Deviceタブ・Actionタブの関数実装
- 例題
  - 定義した機能を実現する方法を検討する
- 課題
  - 動作確認

講義

内部設計③

残りの関数を実装

組込みシステム開発

週	テーマ	内容
7	キッチンタイマーの組立	キッチンタイマーのハードを製作・確認
8 9	要求分析・外部設計	キッチンタイマーに必要な機能を抽出 画面表示、ボタンを押した時の動作を設定 機能の実現手順をフローチャートで図示
10 11 12	内部設計	状態遷移図・状態遷移表の使い方 システムの内部構造を考える 機能の実現手順をより詳細に決める
13 14	実装	プログラムを作る
15	テスト・完成	設計通りにできているか動作検証する

内部設計

- 内部設計
  - 外部設計で定義した機能を、プログラムでどのように実現するか明確にする
  - 開発者の視点から、システム内部でどのような処理を行うかを考える
  - プログラムを共通化できそうな部分を検討

Deviceタブの各関数実装

- 残りの関数を実装
  - スイッチを押したかどうか
    boolean SWIsPressed()
  - ブザーを鳴らす
    void startBuzzer()
  - ブザーを止める
    void stopBuzzer()
  - LEDを点灯する
    void lightOn()
  - LEDを消灯する
    void lightOff()

前回は
SW1IsPressed(), SW2IsPressed()に
分けていたが変更

```
//Deviceタブ（残り）
boolean SWIsPressed(int pin){
  if(digitalRead(pin) == LOW){
    return true;
  } else {
    return false;
  }
}
void startBuzzer(){     //ブザーを鳴らす
  tone(SPKPin, 880);
}
void stopBuzzer(){    //ブザーを止める
  noTone(SPKPin);
}
void lightOn(){       //LEDを点灯する
  digitalWrite(LEDPin, HIGH);
}
void lightOff(){    //LEDを消灯する
  digitalWrite(LEDPin, LOW);
}
```

一部変更
関数名、パラメータ

Actionタブ

- 残りの関数を実装
  - 設定時間を1分増加する
    (60000ミリ秒加算する)
    void addOneMinute()
  - アラームを鳴らす
    void startAlarm()
  - アラームを止める
    void stopAlarm()
  - これら以外は次回以降実装

次回以降実装

```
//Actionタブ
void startTimer(){} //カウントダウンを開始する
void stopTimer(){}  //カウントダウンを停止する


void addOneMinute(){   //設定時間を１分増加する
  time += 60000L;      //timeを60000ミリ秒加算
}

void startAlarm(){     //アラームを鳴らす
  startBuzzer();       //ブザーを鳴らす
  lightOn();           //LEDを点灯する
}

void stopAlarm(){      //アラームを止める
  stopBuzzer();        //ブザーを止める
  lightOff();          //LEDを消灯する
}
```

例題

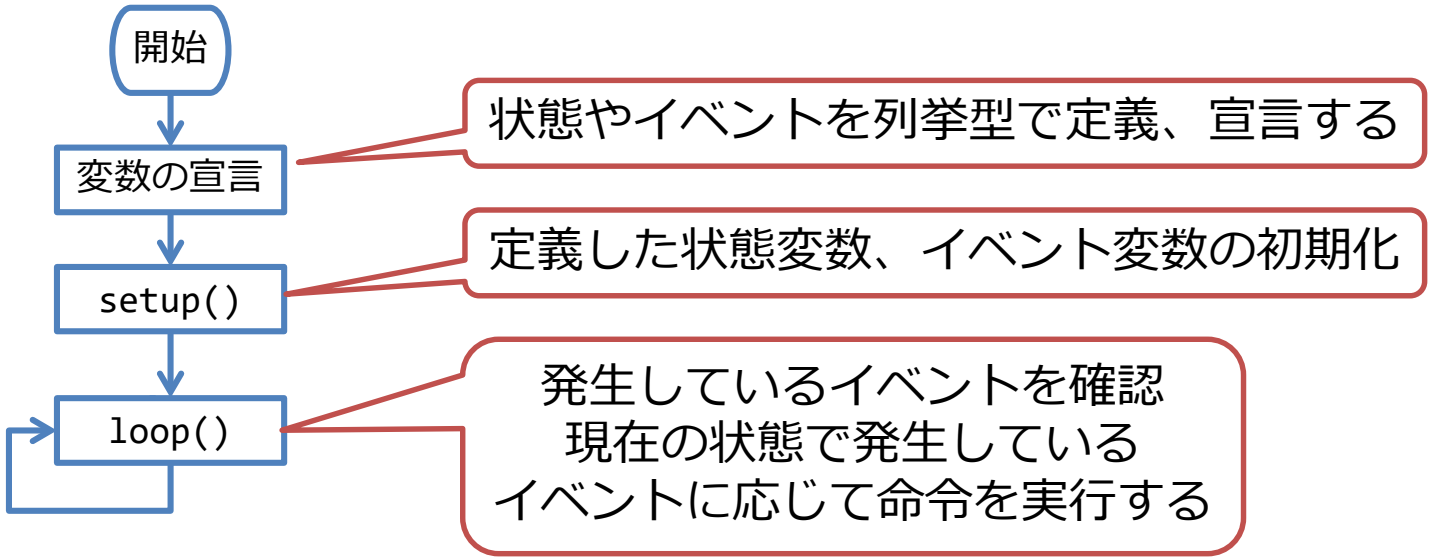
定義した機能を

C言語で実現する

方法を検討する③

実装するためのルール作り③

- 前々回作成したフローチャートをベースに実装
  - 各状態を表す値、各イベントを表す値が必要
  - 今の状態を表す変数、発生しているイベントを表す変数が必要
- メインタブ冒頭で、グローバル変数の宣言
  - 状態、イベントを「列挙型」を使って定義する
  - setup()関数で変数の初期化を行う
- 発生したイベントを確認、状態に応じて命令実行



列挙型

- 整数リストに名前を付けて型として定義
- 定義・型
  - enum タグ名 {定数名1, 定数名2, ・・・} 変数名;
  - 型　enum タグ名

```
//メインタブ
enum State {
  STATE_INIT,            //初期状態
  STATE_SETTING,         //設定中
  STATE_COUNTDOWN,       //カウントダウン中
  STATE_PAUSE,           //一時停止中
  STATE_ALARM            //アラーム鳴動中
} state;

enum Event {
  EVENT_NONE,                        //イベントが発生しない
  EVENT_PUSH_MINUTE_BUTTON,          //分ボタンを押す
  EVENT_PUSH_START_STOP_BUTTON,      //StartStopボタンを押す
  EVENT_TIME_IS_UP                   //残り時間が0になる
} event;
```

状態を表す列挙型
型　enum State
変数名　state

イベントを表す列挙型
型　enum Event
変数名　event

状態を表す列挙型

- 状態を表す型 enum State型
- 定数リスト（各状態を表す定数名）

状態定数名	対応する状態
STATE_INIT	初期状態
STATE_SETTING	設定中
STATE_COUNTDOWN	カウントダウン中
STATE_PAUSE	一時停止中
STATE_ALARM	アラーム鳴動中

状態名は
STATE_状態名と命名

- 変数名 state
  - 初期値：初期状態

イベントを表す列挙型

- イベントを表す型 enum Event型
- 定数リスト（各イベントを表す定数名）

イベント定数名	対応するイベント
EVENT_NONE	イベントが発生していない
EVENT_PUSH_MINUTE_BUTTON	分ボタンを押す
EVENT_PUSH_START_STOP_BUTTON	StartStopボタンを押す
EVENT_TIME_IS_UP	残り時間が0になる

- 変数 event
  - 初期値：イベントが発生していない

イベント名は
EVENT_発生したイベントと命名

setup()で状態・イベント変数の初期化

- 長整数型の変数追加
  - startTime　開始時間
  - remainTime 残り時間
- setup()関数のなかで変数初期化
  - time = 0L;
  - startTime = 0L;
  - remainTimer = 0L;
  - state = STATE_INIT;
  - event = EVENT_NONE;

```
//メインタブ（つづき）
long time;          //設定時間
long startTime;     //カウントダウン開始時間
long remainTime;    //残り時間

void setup() {
  setupDevice();
  time = 0L;
  startTime = 0L;
  remainTime = 0L;

  //状態変数を初期状態に設定する
  state = STATE_INIT;
  //イベント変数を"発生していない"に設定する
  event = EVENT_NONE;
}
//以下省略
```

例題のまとめ

```
//メインタブ KT2018_02
enum State {
  STATE_INIT,
  STATE_SETTING,
  STATE_COUNTDOWN,
  STATE_PAUSE,
  STATE_ALARM
} state;
```

追加

```
enum Event {
  EVENT_NONE,
  EVENT_PUSH_MINUTE_BUTTON,
  EVENT_PUSH_START_STOP_BUTTON,
  EVENT_TIME_IS_UP
} event;
```

追加

```
long time;
long startTime;
long remainTime;
```

追加

```
void setup() {
  setupDevice();
  time = 0L;
  startTime = 0L;
  remainTime = 0L;
  state = STATE_INIT;
  event = EVENT_NONE;
}

void loop() {

  // ここに課題プログラムを追加

}
```

追加

例題のまとめ

```
//Deviceタブ
#include <LiquidCrystal.h>
LiquidCrystal lcd(5, 6, 7, 8, 9, 10, 11);
const int SW1Pin = 2;
const int SW2Pin = 4;
const int LEDPin = 13;
const int SPKPin = 3;
void setupDevice(){
  pinMode(SW1Pin, INPUT);
  pinMode(SW2Pin, INPUT);
  pinMode(LEDPin, OUTPUT);
  lcd.begin(16, 2);
  lcd.clear();
  Serial.begin(9600);
}
void displayString(String str1, String str2){
  lcd.clear();          //画面消去
  lcd.setCursor(0, 0);  //上段左端に移動
  lcd.print(str1);      //文字列str1を表示
  lcd.setCursor(0, 1);  //下段左端に移動
  lcd.print(str2);      //文字列str2を表示
}
```

```
boolean SWIsPressed(int pin){
  if(digitalRead(pin) == LOW){
    return true;
  } else {
    return false;
  }
}
```

追加

```
void startBuzzer(){    //ブザーを鳴らす
  tone(SPKPin, 880);
}
```

追加

```
void stopBuzzer(){   //ブザーを止める
  noTone(SPKPin);
}
```

追加

```
void lightOn(){      //LEDを点灯する
  digitalWrite(LEDPin, HIGH);
}
```

追加

```
void lightOff(){   //LEDを消灯する
  digitalWrite(LEDPin, LOW);
}
```

追加

例題のまとめ

```
//Actionタブ
void startTimer(){}
void stopTimer(){}
void addOneMinute
  time += 60000L;
}
void startAlarm(){
  startBuzzer();
  lightOn();
}
void stopAlarm(){
  stopBuzzer();
  lightOff();
}
void displayTimer(long ms){
  // 省略（前回の解答を参照）
}
```

追加

追加

追加

```
//Eventタブ
boolean MinuteButtonIsPressed(){
  //ここに課題プログラムを追加
}
boolean StartStopButtonIsPressed(){
 //ここに課題プログラムを追加
}
boolean TimeIsUp(){}
enum Event receiveEvent(){}
```

追加

```
//Stateタブ
enum State handleEvent(enum State s,
enum Event e){}
```

追加

ここまで作業した時点で
検証ボタンを押して
エラーがないか確認

課題

動作確認

分ボタンを押すと1分増える

StartStopボタンでリセット

Eventタブに追加

追加

```
//Eventタブ
boolean MinuteButtonIsPressed(){        //分ボタンを押したか
  static boolean val_old = false;       //1回前の測定値を入れる変数
  boolean val = SWIsPressed(SW2Pin);    //現在の測定値
  boolean ret = false;                  //戻り値を入れる変数

  if(val_old == true && val == false){//1回前がtrue，現在がfalseの場合
    ret = true;
  } else {
    ret = false;
  }
  val_old = val;  //測定値更新
  return ret;
}
```

プッシュ式のスイッチは
バウンシングやチャタリングという
現象によって、意図しない入力が
発生することがあるため、スイッチが
ONからOFFへ入力が変化する
タイミングでボタンが押されたと
判断する（教科書P43~45参照）

Eventタブに追加

追加

```
boolean StartStopButtonIsPressed(){  //StartStopボタンを押したか
  static boolean val_old = false;     //1回前の測定値を入れる変数
  boolean val = SWIsPressed(SW1Pin);  //現在の測定値
  boolean ret = false;                //戻り値を入れる変数

  if(val_old == true && val == false){//1回前がtrue，現在がfalseの場合
    ret = true;
  } else {
    ret = false;
  }
  val_old = val;  //測定値更新
  return ret;
}
boolean TimeIsUp(){}
enum Event receiveEvent(){}
```

関数内の変数val_oldは
値を保存しておくために
staticを付けて宣言している

そのため2つの関数の処理内容は
ピン番号を除いて同じであるが、
保存される値が異なるため
分けて関数を定義している

メインタブに追加

3つの関数が
正しく動作するか確認する

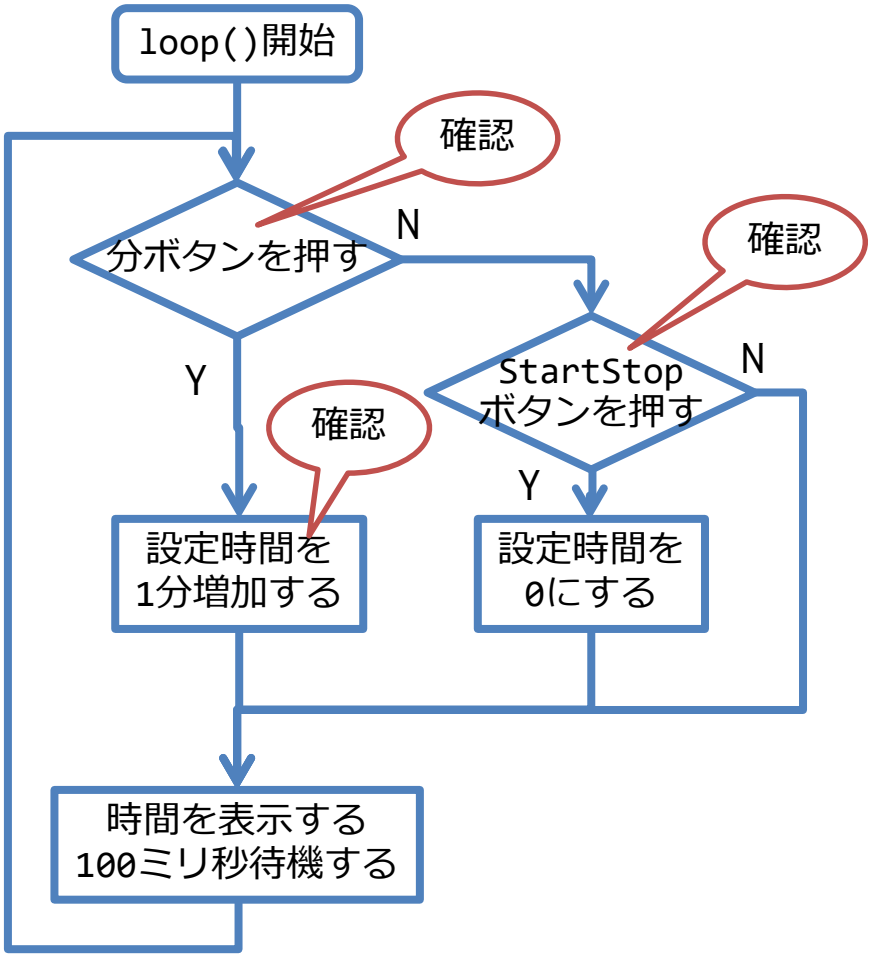
```
//メインタブ


void loop() {
  if( 分ボタンを押す関数 ){
    設定時間が1分増加する関数 ;
  } else if (StartStopボタンを押す関数 ){
    time = 0L;
  }

  displayTimer(time);

  delay(100);
}
```

追加



loop()関数の処理内容

- 分ボタンが押されたら設定時間が1分増加する
- StartStopボタンが押されたら時間が0になる

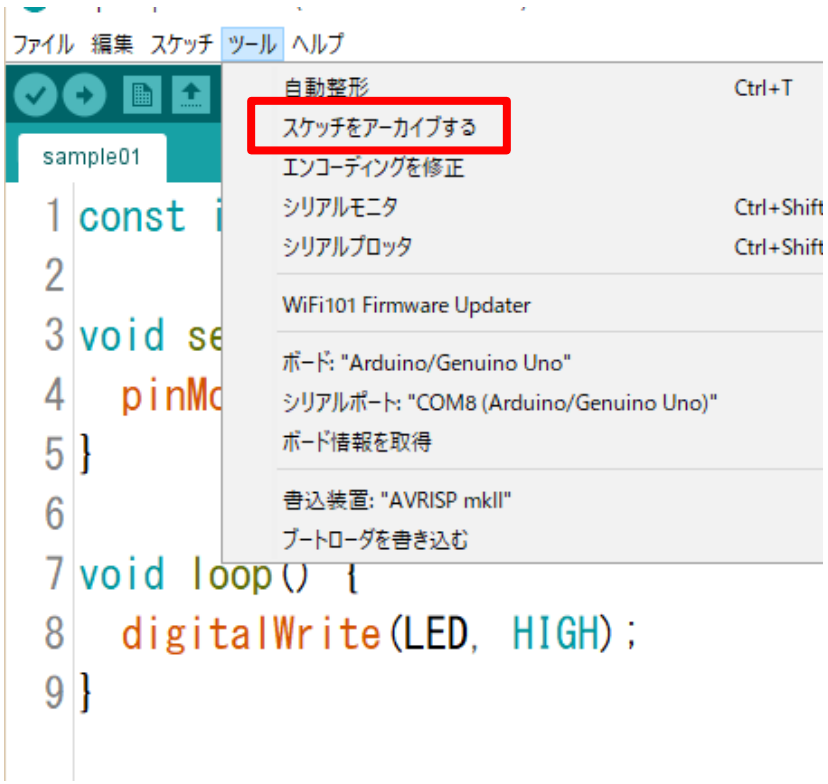
動作確認

- 分ボタン(SW2)を押すと1分増加する
  - 押してから離すタイミングで増加するかどうか確認する
- StartStopボタン(SW1)を押すと時間が0にリセットされる
  - 押してから離すタイミングで0になるかどうか確認する

課題の提出方法

プログラムをアーカイブする

- ツール ->
  スケッチをアーカイブ



- 圧縮ファイル名を指定

課題 提出先・期限

- 圧縮ファイルをメールに添付して提出する
  - 課題プログラム KT2018_02
  - 提出前に動作確認してから提出すること
- 提出先
  - haship@aomori-u.ac.jp
  - 件名：マイコンシステム12週
  - 本文に学籍番号、氏名を書く
- 提出期限
  - 次回授業前まで